cmd\discv4pipeline\main.go

```go
package main

import (
    "crypto/ecdsa"
    "encoding/hex"
    "fmt"
    "log"
    "net"
    "net/netip"
    "time"

    "github.com/ethereum/go-ethereum/cmd/internal/kademlia"
    "github.com/ethereum/go-ethereum/crypto"
    "github.com/ethereum/go-ethereum/p2p/discover/v4wire"
    "github.com/ethereum/go-ethereum/p2p/enode"
    "github.com/ethereum/go-ethereum/p2p/rlpx"
)

func main() {
    // [p2p/enode/localnode.go] 로컬 ENR에 서명할 Alice, Bob 키쌍을 만든다.
    aliceKey := mustKey("alice")
    bobKey := mustKey("bob")

    // [p2p/enode/nodedb.go] ENR/연락처 메타데이터를 저장할 인메모리 노드 DB를 연다.
    aliceDB := mustOpenDB("alice")
    bobDB := mustOpenDB("bob")

    // [p2p/enode/localnode.go] LocalNode는 ENR 키-값 관리와 자동 재서명을 책임진다.
    aliceLocal := enode.NewLocalNode(aliceDB, aliceKey)
    bobLocal := enode.NewLocalNode(bobDB, bobKey)

    // [p2p/enode/localnode.go] 초기 ENR에 fallback IP/UDP를 설정한다.
    aliceAddr := netip.MustParseAddr("192.0.2.1")
    aliceLocal.SetFallbackIP(net.ParseIP(aliceAddr.String()))
    aliceLocal.SetFallbackUDP(30304)
    bobAddr := netip.MustParseAddr("192.0.2.2")
    bobLocal.SetFallbackIP(net.ParseIP(bobAddr.String()))
    bobLocal.SetFallbackUDP(30305)

    // [p2p/enode/localnode.go] Node() 호출 시 최신 ENR에 서명하고 Seq 번호를 갱신한다.
    aliceNode := aliceLocal.Node()
    bobNode := bobLocal.Node()
    fmt.Printf("[ENR] Alice initial seq=%d, UDP=%d\n", aliceNode.Seq(), aliceNode.UDP())
    fmt.Printf("[ENR] Bob   initial seq=%d, UDP=%d\n", bobNode.Seq(), bobNode.UDP())

    // [p2p/discover/v4wire/v4wire.go] discovery v4에서 사용하는 Endpoint를 구성한다.
    aliceEndpoint := v4wire.Endpoint{IP: net.ParseIP(aliceAddr.String()), UDP: 30304, TCP: 30304}
    bobEndpoint := v4wire.Endpoint{IP: net.ParseIP(bobAddr.String()), UDP: 30305, TCP: 30305}

    // [p2p/discover/v4wire/v4wire.go] Ping 패킷을 작성해 Kademlia 본딩을 시도한다.
```

```go
    ping := &v4wire.Ping{Version: 4, From: aliceEndpoint, To: bobEndpoint, Expiration: futureExpiration()}
    pingPacket, pingHash, err := v4wire.Encode(aliceKey, ping)
    if err != nil {
        log.Fatalf("encode ping: %v", err)
    }
    fmt.Printf("[DISC] Alice -> Bob ping (hash=%s)\n", hex.EncodeToString(pingHash))

    // [p2p/discover/v4wire/v4wire.go] Bob은 수신한 패킷을 decode해 발신자 공개키와 payload를 복원한다.
    decoded, fromKey, _, err := v4wire.Decode(pingPacket)
    if err != nil {
        log.Fatalf("decode ping: %v", err)
    }
    fmt.Printf("[DISC] Bob decoded %s from id=%s\n", decoded.Name(), shortID(fromKey.ID()))

    // [p2p/enode/enode.go] Bob은 Alice의 ENR을 enode.Node로 복원해 DB에 기록한다.
    bobDB.UpdateNode(aliceNode)
    bobDB.UpdateLastPingReceived(aliceNode.ID(), bobAddr, time.Now())
    fmt.Printf("[ENR] Bob stored Alice seq=%d in DB\n", bobDB.Node(aliceNode.ID()).Seq())

    // [p2p/enode/localnode.go] Alice가 외부 IP/포트를 학습했다고 가정하고 ENR을 갱신한다.
    aliceLocal.SetStaticIP(net.ParseIP("198.51.100.77"))
    aliceLocal.SetFallbackUDP(40404)
    updatedAlice := aliceLocal.Node()
    fmt.Printf("[ENR] Alice updated seq=%d, UDP=%d (should propagate)\n", updatedAlice.Seq(), updatedAlice.UDP())
    aliceDB.UpdateNode(updatedAlice)
    fmt.Printf("[ENR] Alice DB now holds seq=%d\n", aliceDB.Node(updatedAlice.ID()).Seq())

    // [cmd/internal/kademlia/table.go] 단순 라우팅 테이블에 Bob을 삽입해 discovery 이후 상태를 재현한다.
    table := kademlia.NewTable(updatedAlice.ID(), 16)
    inserted := table.AddNode(kademlia.Node{
        ID:     bobNode.ID(),
        Pubkey: v4wire.EncodePubkey(&bobKey.PublicKey),
        IP:     bobAddr,
        UDP:    uint16(bobNode.UDP()),
        TCP:    30305,
        Label:  "Bob",
    })
    fmt.Printf("[TABLE] inserted Bob into Alice view? %v\n", inserted)
    fmt.Println("[TABLE] 실제 p2p/discover/table.go는 여기서 revalidation/replacements를 병행한다.")

    // [p2p/discover/v4wire/v4wire.go] Bob이 pong을 보내 본딩을 마무리한다.
    pong := &v4wire.Pong{To: aliceEndpoint, ReplyTok: pingHash, Expiration: futureExpiration()}
    pongPacket, _, err := v4wire.Encode(bobKey, pong)
    if err != nil {
        log.Fatalf("encode pong: %v", err)
    }
    if _, _, _, err := v4wire.Decode(pongPacket); err != nil {
```

```go
        log.Fatalf("decode pong: %v", err)
    }
    fmt.Println("[DISC] Bob -> Alice pong acknowledged")

    // [p2p/rlpx/rlpx.go] discovery에서 얻은 공개키로 RLPx 핸드셰이크를 수행한다.
    dialConn, listenConn := net.Pipe()
    dialer := rlpx.NewConn(dialConn, &bobKey.PublicKey)
    listener := rlpx.NewConn(listenConn, nil)

    handshakeErr := make(chan error, 1)
    go func() {
        _, err := listener.Handshake(bobKey)
        handshakeErr <- err
    }()

    remotePub, err := dialer.Handshake(aliceKey)
    if err != nil {
        log.Fatalf("dialer handshake failed: %v", err)
    }
    if err := <-handshakeErr; err != nil {
        log.Fatalf("listener handshake failed: %v", err)
    }
    fmt.Printf("[RLPx] handshake complete with remote=%s\n", shortID(pubkeyToID(remotePub)))

    // [p2p/rlpx/rlpx.go] 핸드셰이크 이후에는 암호화된 프레임을 주고받을 수 있다.
    const msgCode = 0x10
    payload := []byte("hello over rlpx")
    if _, err := dialer.Write(msgCode, payload); err != nil {
        log.Fatalf("rlpx write: %v", err)
    }
    code, data, _, err := listener.Read()
    if err != nil {
        log.Fatalf("rlpx read: %v", err)
    }
    fmt.Printf("[RLPx] listener received code=0x%x data=%q\n", code, string(data))

    _ = dialer.Close()
    _ = listener.Close()
}

func futureExpiration() uint64 { // [p2p/discover/v4wire/v4wire.go] v4 패킷 만료시간 계산과 동일한 방식.
    return uint64(time.Now().Add(10 * time.Second).Unix())
}

func shortID(id enode.ID) string { // [p2p/enode/enode.go] 터미널 출력용 short ID 포맷.
    return hex.EncodeToString(id[:4])
}

func pubkeyToID(pub *ecdsa.PublicKey) enode.ID { // [p2p/enode/enode.go] 공개키에서 ENR v4 ID를 파생한다.
    encoded := v4wire.EncodePubkey(pub)
    return encoded.ID()
```

```go
}

func mustKey(label string) *ecdsa.PrivateKey { // 보일러플레이트: 키 생성 실패 시 패닉.
    key, err := crypto.GenerateKey()
    if err != nil {
        log.Fatalf("generate %s key: %v", label, err)
    }
    return key
}

func mustOpenDB(label string) *enode.DB { // 인메모리 ENR DB 헬퍼.
    db, err := enode.OpenDB("")
    if err != nil {
        log.Fatalf("open %s db: %v", label, err)
    }
    return db
}

```